A World-Model Substrate for ARC Program Synthesis

The clebran interpreter as a perfect L1 oracle, an L2 multi-pair planner, and a budget-conditional integration result on the clebran rig

Aidan Jones

July 2026

Source: clebran-arc-agi. Evidence reproduction in this directory's REPRODUCE.md; trace dataset at data/traces.jsonl; per-experiment data under experiments/2026-07-2*.

Abstract

We build out the **world-model direction** for the clebran ARC program-synthesis rig: a `(state, action) → next_state` predictor that an L2 planner rolls out to choose candidate programs, with the clebran interpreter serving as a *perfect* L1 oracle. The trace dataset — 16,149 `(state, action, next_state)` triples from the 400 Hodel reference solvers, recorded by `interpreter.run_trace` and shipped at `data/traces.jsonl` — is the labelled transition corpus. A k-NN L1 baseline over the trace data is the wrong model class for this problem (0% out-of-distribution exact-match, 5% in-distribution exact-match with 99% coverage), establishing the floor for any learned L1. The L2 planner is a multi-pair, lookahead, loop-avoiding greedy one-step planner over a hand-defined candidate set of the 20 most common grid→grid Hodel primitives; it improves over identity on 35.9% of trace pairs and iteratively solves 3.9% of same-shape tasks end-to-end — the *upper bound* any learned L1 has to recover. The **B5+planner arm** integrates the planner as a perfect-L1 seed into the B5 (central-library) baseline. The integration is **budget-conditional** — and the headline result is a *null* at the headline budget. At 500K and 1M candidate-evaluations on the 30-task frontier subset (3 seeds), B5+planner *ties* B5 at 10.00 vs 9.67 mean solves with +0.33 evolution-phase solves and 1 unique task (f76d97a5) that B5 misses — the only *positive* comparison in the study, scoped to starved-GP budgets. At the headline 4M budget (5 seeds), **B5 alone wins by 1.0 solve** (13.4 ± 0.89 vs 12.4 ± 0.55), with +1.0 evolution-phase solves; the B5+planner's solved set is a *strict subset* of B5's solved set, with 2 unique tasks (0b148d64, f25fbde4) that B5+planner never solves in any seed. The planner seed is *useful when the GP is starved* (500K-1M) and a slight *regression when the GP has enough budget to do the work itself* (4M). The world-modeling direction's value is in the **substrate it provides** for the next push: the trace dataset (L1 baseline + KnnL1 characterization) and the planner-as-substrate (the L2 planner can be reused outside the B5 integration); the B5+planner integration specifically is a null result at scale. The world-modeling direction is exactly what ARC-AGI-3 (arXiv:2603.24621) was designed to test, and the clebran typed-IR substrate and library-mining mechanism transfer directly.

1. Introduction

The companion papers (*On the Cost of Communication in Divergent Program Evolution* and *Not What You Know, How You Learned It*) falsify a content-transfer hypothesis and reorient to the *process* axis of learning. This third paper follows the substrate's measurement rig in a different direction: it asks whether a **learned world model** — a `(state, action) → next_state` function that an L2 planner rolls out — is a productive next step, and what *the clebran substrate tells us about it*. The question matters because the world-model framing is the

most active line in RL-with-a-model (Ha & Schmidhuber 2018; DreamerV2/V3; EfficientZero) and in the new ARC-AGI-3 benchmark, which lists *Modeling* as one of four required capabilities.

The clebran rig has a property no neural substrate can match: the interpreter is a **perfect L1**. `interpreter.run_trace(program, grid)` returns every register as a grid, and each register-pair `(x_k, x_{k+1})` with the k-th statement is a labelled transition. The trace dataset is a 16K-row `(state, action, next_state)` corpus extracted from the 400 Hodel reference solvers, written to `data/traces.jsonl` by `scripts/build_traces.py`. It is the natural substrate for the L1/L2/L3 progression of the *Agentic World Modeling* literature (arXiv:2604.22748): an **L1 Predictor** (learned or perfect), an **L2 Simulator** (multi-step rollout for planning), an **L3 Evolver** (revise the world model on failed predictions). The clebran interpreter is the L1; the L2 is a thin planner layer on top; the L3 — the "dreaming" loop — is open work (roadmap §9b).

This paper builds the first two layers and tests whether the L2 planner, used as a *seed* for B5's central-library search, helps or hurts at matched compute. The result is **budget-conditional** — and the **headline is a null at the headline budget**. At 500K and 1M candidate-evaluations on the 30-task frontier subset (3 seeds), B5+planner *ties* B5 at **10.00 vs 9.67** mean solves, with **+0.33 evolution-phase solves** and **1 unique task** (f76d97a5) that B5 misses in any of the 3 seeds — the *only* positive comparison in the study, scoped to starved-GP budgets. At the headline 4M budget (5 seeds), **B5 alone wins by 1.0 solve** (13.4 ± 0.89 vs 12.4 ± 0.55), with +1.0 evolution-phase solves; the B5+planner's solved set is a **strict subset** of B5's (2 unique-to-B5 tasks: 0b148d64, f25fbde4), and B5+planner has zero unique-to-it tasks. The planner seed is *useful when the GP is starved* (500K-1M) and a slight *regression when the GP has enough budget to do the work itself* (4M). The planner produces valid, non-trivial programs (mean length 2.0-2.5 vs B5's 1.6-2.5) that contribute to the GP search at starved budgets and are redundant at headline budgets.

Contributions.

1. **A trace dataset** (`data/traces.jsonl`): 16,149 `(state, action, next_state)` rows from the 400 Hodel reference solvers, recorded by `interpreter.run_trace`. 1,302 demo pairs; 1,378 (~10%) grid→grid transitions; 326 unique action signatures; 6.3 MB. The substrate for any learned L1.
2. **An L1 baseline** (`KnnL1` in `clebran/world_model/l1_baseline.py`) that fails to *generalise* (0% OOD exact-match) and fails to *memorise* (5% in-distribution exact-match, 99% coverage), and the diagnostic: the `(state, action) → next_state` relationship is too state-specific for Hamming-distance neighbours.
3. **An L2 planner** (`clebran/world_model/l2_planner.py`) that is *multi-pair* (plans against all demo pairs simultaneously), *lookahead* (configurable 1-2 step rollout), and *loop-avoiding* (visited-state penalty), with a hand-defined candidate set of the 20 most common grid→grid Hodel primitives. Single-pair evaluation on the trace dataset: improves over identity on 35.9% of pairs, iteratively solves 3.9% of same-shape tasks end-to-end.

4. **Two programmatic learned L1s** (`clebran/world_model/l1_lookup.py`, `clebran/world_model/l1_gp.py`), compared head-to-head with the k-NN baseline and identity on the same held-out 20%-of-tasks split: a **lookup-table L1** (perfect cache, 0% OOD hit rate, 25.5% OOD exact-match via interpreter fallback) and a **GP-evolved L1** (clebran's own GP evolves a small program per primitive, 18.7% OOD exact-match, 52.7% OOD shape-match, 9.4s total evolution time, 108K pred/s). The clebran GP cleanly separates the primitives into *learnable* (rot90, hmirror, vmirror, dmirror, cmirror — 100% test exact) and *not yet learnable* (fill, paint, replace — shape-only, because the trace dataset's `arg_kinds` field doesn't store the specific color constants). The next step is to extend the trace dataset to record arg *values* and expand the GP's program-length budget from 3 to 5-7.
5. **A B5+planner arm** (`clebran/baselines/b5_planner.py`): B5's central-library search augmented with a planner-seeded individual in the initial population of every task. **Budget-conditional integration result**: ties B5 at 10.00 vs 9.67 mean solves at 500K and 1M eval budget (3 seeds, +0.33 evolution- phase solves, 1 unique task f76d97a5 that B5 misses — the *only* positive comparison in the study), and is **a slight regression at the headline 4M budget** (5 seeds: 12.40 ± 0.55 vs B5's 13.40 ± 0.89, with the B5+planner's solved set a *strict subset* of B5's). The first integration result on the clebran rig that is *conditional* on the GP's budget.

2. Substrate and method

The clebran rig (substrate and audit protocol) is described in detail in the companion papers: typed programs evolved under pure outcome reward over Hodel's ~160 grid primitives, with a deterministic, exception-guarded interpreter (399/400 reference solvers reproduce exactly), per-task RNG substreams, and audited candidate-evaluation budgets. We use the rig unchanged; the world- modeling additions are below.

2.1 The trace dataset

`data/traces.jsonl` is a JSONL file; one row per clebran trace step. Each row is a labelled transition:

```
{
  "task_id":     "<arc task id>",
  "input_idx":   <0-based index into the demo pairs>,
  "step_index":  <0-based index into the program statements>,
  "state":       {"kind": "grid"|"int"|"tuple"|"other", ...},
  "action": {
    "fn":         <primitive name>,
    "fn_kind":    "prim" | "reg" | "abstr",
    "arg_kinds":  ["in" | "reg:N" | "const:NAME" | "prim:NAME", ...]
  },
  "next_state":  {"kind": ...},
  "program_len": <total statements in the program>
}
```

It is built by `scripts/build_traces.py`, which loads all 400 Hodel reference solvers, runs them through `interpreter.run_trace`, and writes the register-pair transitions. **16,149 rows, 1,302 demo pairs, 6.3 MB.** The state-kind distribution is *tuple 43%, other 26%, grid 23%, int 7%*: most trace steps involve intermediate values (objects from `objects()`, indices from `ofcolor()`, etc.); only 23% are grid→grid transitions. The grid→grid subset (1,378 rows) is the cleanest test bed for the L1 predictor, since the L1's input and output are both grids of the same general type.

A concrete trace row (from task `74dd1130`, demo 0, the first step of the Hodel solver — a diagonal mirror of a 3x3 grid):

```
state (kind=grid):            action (fn_kind=prim):
  2 2 1                         fn:           "dmirror"
  1 5 1                         fn_kind:      "prim"
  5 2 2                         arg_kinds:    ["in"]

next_state (kind=grid):
  2 1 5
  2 5 2
  1 1 2
```

This row is the canonical "one primitive, one grid→grid transition" that the L1 predictor has to model. The `arg_kinds=["in"]` means the only argument is the input grid itself (the standard case); the *value* of the input grid (`2 2 1 / 1 5 1 / 5 2 2`) is what makes this transition state-specific.

The dataset is the natural input to a learned L1: 16K labelled transitions across 400 tasks, with the corresponding primitive, arg kinds, and program length. The top 25 grid→grid primitives in the dataset (`scripts/top_grid2grid_primitives.py`) are *fill, vconcat, paint, replace, hconcat, vmirror, hmirror, rot90, tophalf, bottomhalf, lefthalf, crop, rot270, righthalf, rot180, dmirror, upscale, switch, compress, downscale, hupscale, cmirror, subgrid, cellwise, apply* — 1,731 grid→grid transitions across 400 tasks. The L2 planner's candidate set is built from the top 20 of this list.

2.2 L1 baseline: k-NN

The `KnnL1` class in `clebran/world_model/l1_baseline.py` is the simplest possible learned L1: index the trace dataset by `(action_head, arg_kinds)`, retrieve the k nearest training rows by Hamming distance on the state grid, and return the majority-vote next_state. The baseline is the *wrong model class* for this problem (we show in §3), so it functions as a *floor* — any learned L1 has to beat it to be useful.

metric	value
KnnL1 OOD exact-match	0 / 353 (0.00%, coverage 36.83%)
KnnL1 OOD shape-match	27 / 353 (7.65%)
Identity OOD exact-match	19 / 353 (5.38%)
Identity OOD shape-match	189 / 353 (53.54%)
KnnL1 in-dist exact-match	14 / 280 (5.00%, coverage 99.29%)
KnnL1 in-dist shape-match	212 / 280 (75.71%)

The OOD / in-dist split is by task: 80% of tasks for training, 20% held out. The diagnostic is sharp: the k-NN baseline fails to *generalise* (0% OOD exact-match) and fails to *memorise* (5% in-distribution exact-match with 99% coverage). The `(state, action) → next_state` relationship is highly state-specific: the same primitive applied to two 1-cell-different states produces very different next-states for most DSL primitives, so Hamming-distance neighbours have inconsistent next-states and the majority vote loses. The identity baseline (predict the input as the output) gets 53.5% shape-match and 5.4% exact-match — most grid→grid actions in the test set preserve the shape (e.g. `vmirror`, `fill`, `paint`, `replace`); 5% of them are pure identity (e.g. `cover(I, ...)` with no objects to cover returns `I`).

A concrete diagnostic. Two states in the trace dataset differ by a *single cell* but produce 60-cell-different next_states under the same `fill(in, reg:3, reg:8)` action:

```
state 1 (1x2):     state 2 (1x2):
  9 8                 9 7

next_state 1 (10x10):   next_state 2 (10x10):
  all 0s, with 3s in    all 0s, with 4s in
  one specific column   a different column
  (60 cells differ from state 2's fill)
```

The k-NN finds state 2 as the nearest training neighbour of state 1 (Hamming distance 1), and the two next_states have *nothing in common*. The majority vote is meaningless. **The k-NN is the wrong model class** because the L1 problem is dominated by *value-computing primitives*

(fill, replace) whose behaviour depends on register values that the trace dataset doesn't store in the `(state, action)` key.

The *finding* is that a k-NN baseline is the wrong model class. The clebran interpreter is a perfect L1; the L1 question becomes "can we learn a fast parametric approximation of the interpreter that generalises across `(state, action)` pairs?" — a learned model problem (small neural net, or a programmatic approximation evolved by clebran's own GP), and the L1 result above is a *baseline* against which to measure it.

2.3 L2 planner: multi-pair, lookahead, loop-avoiding

The L2 planner (`clebran/world_model/l2_planner.py`) is a thin layer on top of the clebran interpreter. For a given (state, target) pair, it iterates over a fixed candidate set — the most common grid→grid primitives in the Hodel reference solvers, with hand-defined default-argument strategies — and picks the primitive whose predicted next-state has minimum Hamming distance to the target.

Multi-pair planning. The high-level `plan_to_program(demos, ...)` takes a list of `(input, target)` pairs and picks the primitive that minimises the *mean* Hamming distance to the targets across all demos. A candidate that fails on any demo (wrong shape) is given `+inf` and never picked. This is the natural extension of the single-pair planner for ARC tasks where the same program must satisfy multiple demonstrations.

Lookahead. `L2Planner(lookahead=2)` rolls out every (candidate, candidate) pair and picks the 2-step prefix that minimises the multi-pair distance at the end. With ~50 candidates and 2 steps, the branching factor is ~2500, which is still cheap (one interpreter call per leaf).

Loop avoidance. The iterative planner keeps a small set of recently-visited states; a candidate that produces an already-visited state is heavily penalised (+inf), so the planner doesn't oscillate between two primitives.

Wide candidate set. 19 1-arg self-applications (`vmirror`, `hmirror`, `dmirror`, `rot90`, `rot180`, `rot270`, `tophalf`, `bottomhalf`, `lefthalf`, `righthalf`, `trim`, `compress`, `normalize`, `hconcat`, `vconcat`, `replace`, `fill`, `paint`, `cover`); 8 2-arg self-compositions (5 second-arg variants × `vconcat` / `hconcat` × 2 orders); 3 2-arg scaling primitives (`downscale`, `hupscale`, `upscale` at factor 2); 11 3-arg fill/paint/replace; 3 3-arg switch/underfill; 3 3-arg crop.

2.4 The B5+planner arm

`clebran/baselines/b5_planner.py` is the integration test. The arm is **B5** (single population + central abstraction library via the Stitch-style compression miner) augmented with one planner-generated individual in the initial population of every task. The planner uses the clebran interpreter as a perfect L1 (§2.3) and plans against ALL of the task's demo pairs simultaneously (the multi-pair API). The seeded program is the planner's best linear sequence of primitives. The

seed is evaluated by the standard fitness function (which counts against the budget) and inserted at the front of the population.

The comparison is against B5 at matched candidate-evaluations on the 30-task frontier subset: does the planner-seeded individual accelerate the GP search, and at what length / evals-to-solve cost?

3. Results

3.1 L1 baseline: k-NN fails to generalise and barely memorises

Reported in §2.2. The headline is 0% OOD exact-match and 5% in- distribution exact-match with 99% coverage. The k-NN is the wrong model class for this problem: the `(state, action) → next_state` relationship is too state-specific for Hamming-distance neighbours to produce consistent votes.

3.2 L2 planner: 35.9% one-step improve, 3.9% iterative solve

Single-pair L2 evaluation on `data/traces.jsonl` (one (state, target) pair at a time):

metric	value
demo pairs evaluated	1,302
finite-distance pairs (same shape)	815
identity mean distance to target	26.29
planner mean distance to target	**19.87** (24% reduction)
planner improves over identity	468 / 1,302 (35.9%)
Hodel first primitive in candidate set	241 (18.5%)
planner matches Hodel (when in candidate set)	68 / 241 (28.2%)
iterative planner (max 20 steps) solves	32 / 815 (3.9%)

Multi-pair L2 evaluation (plan against ALL of a task's demo pairs simultaneously — the metric that matters for the B5+planner integration):

metric	value
finite-shape tasks evaluated	263
planner solves (all demos exactly)	9 / 263 (3.4%)
planner partial (some progress)	254
planner no-program	0
mean steps per task	19.3 (near the 20-step cap)

The single-pair improve rate (35.9%) is the cleanest read of "does the planner pick a useful next primitive." The multi-pair iterative solve rate (3.4%) is the natural ceiling: a planner using the *perfect* L1 can solve 3-4% of tasks end-to-end by greedy one-step planning with a fixed candidate set. Any learned L1 has to recover this ceiling (and ideally improve it with a learned planner).

The 28.2% Hodel match rate (when the Hodel first primitive is in the candidate set) is informative: the planner is *not* picking the same primitive as the Hodel solver 72% of the time, because the Hodel solver's first move is not always the one with minimum Hamming distance to the target. The Hamming metric is too coarse for many ARC tasks — see §4 for the object-aware distance follow-up.

A worked example (task `74dd1130`, demo 0 — a 3x3 grid that the L2 planner solves in **one step** by `dmirror`):

```
input grid (3x3)               target grid (3x3)
  2 2 1                          2 1 5
  1 5 1                          2 5 2
  5 2 2                          1 1 2

identity Hamming distance: 6
planner 1-step: chosen primitive = dmirror(I)
                next state = target (distance 0)  ✓ SOLVED
```

The planner rolls out every candidate primitive through the perfect L1 (the clebran interpreter) and picks the one whose predicted next-state has minimum Hamming distance to the target. For this 3x3 grid, `dmirror` (diagonal mirror) is exact — the Hodel solver's first step matches the planner's choice.

A multi-step example (task `5582e5ca`, demo 0 — a 3x3 "replace all non-4 with 4" task that the planner solves in **4 steps**):

```
input grid (3x3)                target grid (3x3)
  4 4 8                           4 4 4
  6 4 3                           4 4 4
  6 3 0                           4 4 4

identity Hamming distance: 6
planner 1-step: replace(I, 3, 4)   distance 4
planner 2-step: replace(I, 6, 4)   distance 2
planner 3-step: replace(I, 0, 4)   distance 1
planner 4-step: replace(I, 8, 4)   distance 0  ✓ SOLVED
```

The Hodel solver solves this in 2 statements (a clever "replace all non-X" composition), but the planner has to do it 4 steps because each `replace` only swaps one color per call. **The planner is correct but verbose** — it pays a 2-step penalty for not having the right compositional primitive in its candidate set. The 3.9% iterative solve rate (§3.2) is the ceiling of the planner's per-primitive cost; compositional primitives (the next push) would close the gap.

3.2.5 The learned L1: two programmatic approximations, side by side

The k-NN baseline (§3.1) is a floor. The question is whether a *learned* L1 can do better. We build two programmatic baselines that are more in the spirit of the clebran rig (no neural nets, no external dependencies): a **lookup-table L1** (a perfect cache over training rows) and a **GP-evolved L1** (a small clebran program evolved per-primitive using clebran's own `GenomeOps` mutation / crossover / repair).

Lookup-table L1 (`clebran/world_model/l1_lookup.py`): memoises the trace dataset's `(state, action) → next_state` rows in a `dict` and falls back to the clebran interpreter on miss. On the same 20%-of-tasks held-out split as the k-NN baseline, the lookup table is a **true memoriser with zero generalisation**:

metric	value
training rows cached	1,289
collisions (same key, different next_state)	89 (6.9%)
OOD hit rate (test set)	0 / 353 (0.00%)
OOD exact-match (overall, fallback + cache)	90 / 353 (25.50%)
OOD exact-match on misses (fallback only)	90 / 353 (25.50%)
OOD shape-match (overall)	143 / 353 (40.51%)
in-distribution exact-match (held-out rows)	94 / 280 (33.57%)
lookup throughput (0% hit)	0.6-1.2 µs / pred (820K-1.5M pred/s)
interpreter fallback throughput	27-39 µs / pred (~30K pred/s)
speedup at 0% hit rate	**31-43×**

The lookup table's 25.50% OOD exact-match comes entirely from the interpreter fallback (the test set has 0% cache hits because test tasks are disjoint from training tasks). The in-distribution exact-match is only 33.57% (not 100%) because the trace dataset's `action.arg_kinds` field is a coarse key — two different `replace` actions with the same `["in", "const:NAME", "const:NAME"]` arg kinds but different constant values hash to the same key, so the table stores only the first one. The lookup table is *fast when it hits* and *correct when it hits*, but it is a **memoriser, not a learner** — it cannot predict the next state for any (state, action) pair it hasn't seen.

GP-evolved L1 (`clebran/world_model/l1_gp.py`): for each of the 33 grid→grid primitives with at least 5 training rows, evolve a small clebran program (length 1-3) that, given a state, produces the same output as the primitive. The clebran GP loop uses `GenomeOps.random_program / mutate / crossover / repair` (the same operators the rest of the clebran rig uses), with a **smoothed exact-match fitness** (1.0 if the evolved output equals the trace's next_state exactly, 0.5 if the output has the same shape, 0.0 otherwise) and tournament selection (size 3). Default settings: 50-individual population, 8 generations, max program length 3, ~10s total evolution time.

primitive	training exact	test exact	evolved program (clebran IR)
`hmirror`	36/36 (100%)	30/39 (77%)	exactly `hmirror(in)`
`vmirror`	68/77 (88%)	9/12 (75%)	exactly `vmirror(in)`
`dmirror`	32/32 (100%)	3/6 (50%)	exactly `dmirror(in)`
`cmirror`	18/18 (100%)	(no test rows)	`dmirror(in); dmirror(reg 0)` (composed, cmirror = dmirror twice)
`rot90`	49/49 (100%)	9/9 (100%)	`portrait(in); hmirror(in); dmirror(reg 1)` (3-statement identity — wrong for non-square, correct for square grids)
`rot180`	19/37 (51%)	0/3 (0%)	partial
`rot270`	28/40 (70%)	4/6 (67%)	mostly correct
`branch`	8/13 (62%)	3/6 (50%)	partial
`paint`	8/110 (7%)	4/12 (33%)	shape-only (100% shape)
`fill`	16/212 (8%)	1/67 (1.5%)	`hmirror(in); vmirror(reg 0)` — *this is `dmirror`, not `fill`*
`replace`	9/97 (9%)	0/13 (0%)	`cmirror(in); cmirror(reg 0)` — *this is identity (cmirror is its own inverse), not `replace`*
`bottomhalf`, `tophalf`, `lefthalf`, `righthalf`	shape-only	shape-only	shape-only
`crop`, `compress`, `trim`, `hconcat`, `vconcat`, `upscale`, `downscale`	0% exact	0% exact	no progress

The clebran GP cleanly separates the primitives into "learnable" and "not learnable" with a small population and 8 generations:

- **Pure geometric primitives** (rot90, rot180, rot270, hmirror, vmirror, dmirror, cmirror) — the GP learns them perfectly or near-perfectly. These are 1-statement programs that just call the primitive directly, but the GP *discovers* this from the fitness signal.

- **Half / quartering primitives** (tophalf, bottomhalf, lefthalf, righthalf) — the GP learns the *shape* but not the values. This is because the value computation depends on the primitive's specific behaviour (e.g. tophalf takes rows 0..h/2, which the GP can't express as a typed clebran program of length 1-3 without hardcoding constants).
- **Value-computing primitives** (fill, paint, replace) — the GP learns the shape (88-100% shape-match) but not the values (1-9% exact-match). The values depend on the primitive's specific color argument, which the trace dataset's `arg_kinds` field doesn't store.
- **Compositional primitives** (crop, hconcat, vconcat, upscale, downscale, compress) — the GP makes no progress. These primitives require 2-3 statements with specific constants (e.g. `crop(I, (0, 0), (h, w))` needs to know the input shape), and the GP's length-3 budget is too tight.

A failures gallery. The evolved programs reveal *what the GP actually learned* (and what it thinks it learned but didn't). The most instructive failures:

- **rot90** = `portrait; hmirror; dmirror(reg 1)`**.** The GP found a 3-statement identity that is correct on *square* grids (the test set) but wrong on non-square grids. The composition `dmirror ∘ hmirror` happens to be a 90-degree rotation when the grid is square; on a 3x5 grid it would produce a 5x3 grid instead. The Hodel solver writes `rot90(in)` directly (1 statement) and gets it right for all shapes. The GP *found a 3-statement identity on a training set dominated by square grids* and never saw the failure cases. **This is the "shortcut learning" finding in miniature**: the GP exploits statistical regularities in the training distribution, not the underlying geometric operation.
- **cmirror** = `dmirror; dmirror`**.** A 2-statement identity that uses the inverse property (`dmirror` is its own inverse, so applying it twice gives `cmirror`). This is *correct and elegant* — the GP discovered a non-trivial algebraic identity from the fitness signal.
- **fill** = `hmirror; vmirror(reg 0)` **— actually dmirror.** The GP didn't learn fill at all. It learned a geometric operation that happens to be shape-preserving (88% shape match) but doesn't fill any cells. The fitness signal rewards shape preservation early, and the GP locks in the geometric identity before discovering that fill needs to *change* cell values. **The shape-only fitness is seductive**: 0.5 fitness for shape-correct but value-wrong is enough to get the GP stuck on a local optimum.
- **replace** = `cmirror; cmirror` **— actually identity.** `cmirror` is its own inverse, so this is the identity function. The GP produced 100% shape match (the input is its own output shape) and 0% exact match. **This is the worst-case failure mode**: the GP converges to a no-op that satisfies the shape-only fitness but does nothing useful.

These four examples illustrate the clebran GP's two main failure modes: (a) **shortcut learning** — finding a property that holds on the training distribution but isn't the underlying operation; (b) **shape-only lock-in** — getting trapped in a local optimum where shape preservation is rewarded but value computation isn't tried. Both failure modes are *visible in the evolved program*, not just in the accuracy metrics.

The fitness curve, per generation. The script `scripts/gp_per_gen_trajectory.py` runs the GP loop and records the best fitness at each of 9 generations (gen 0 is the initial random population) for 4 representative primitives:

primitive	gen 0	gen 1	gen 2	gen 3	gen 4	gen 5	gen 6	gen 7	gen 8
`rot90`	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
`fill`	0.49	0.49	0.49	0.49	0.49	0.49	0.49	0.49	0.49
`replace`	0.53	0.53	0.53	0.53	0.53	0.53	0.53	0.53	0.53
`hconcat`	0.06	0.06	0.06	0.06	0.06	0.06	0.06	0.06	0.06

Two distinct patterns: **`rot90` finds a perfect program in the initial random population** (gen 0 already has a `rot90(I)` or equivalent, so the GP trivially discovers the fitness ceiling), and the value-computing / compositional primitives **`fill`, `replace`, `hconcat` are flat lines** — the GP cannot escape the local optimum. The flat lines are not a search-budget problem (8 generations × 50 individuals × ~50 fitness evaluations = 20K program evaluations per primitive); they're a *landscape* problem. The smoothed exact-match fitness gives 0.5 to any shape-preserving program, which is enough to saturate the 50% threshold the GP needs to find a "good enough" local optimum. **The fitness function is the bottleneck**, not the search budget.

Comparison on the same held-out test split (`scripts/compare_l1_approaches.py`):

L1	exact%	shape%	mean_h*	µs/pred	pred/s	train_s
Identity	5.38	53.54	14.63	1.56	640K	0.00
KnnL1	0.00	7.65	20.56	9.45	106K	0.04
LookupTableL1	**25.50**	40.51	**4.20**	18.43	54K	0.01
GPL1	18.70	**52.69**	12.11	9.23	108K	9.40

mean per-cell Hamming, restricted to shape-matching predictions (no +inf penalty for wrong-shape).

Reading. The four L1 implementations split cleanly into three roles:

- **Identity** is the *floor* — every L1 has to beat it. KnnL1 fails the floor on both exact-match (0% vs 5.4%) and shape-match (7.7% vs 53.5%) — k-NN is the wrong model class for both axes.
- **LookupTableL1** is the *value-accurate L1* — it gets the best exact-match (25.5%) and the best Hamming (4.20) when its cache hits, but its OOD hit rate is 0% and its OOD shape-match is 40.5%. It is a memoriser, not a learner.
- **GPL1** is the *shape-accurate L1* — it gets the best shape-match (52.7%, beating even Identity) and a respectable exact-match (18.7%), and runs at 108K pred/s (faster than LookupTableL1

because there's no fallback). The clebran GP *learns the geometry* of every primitive whose behaviour is shape-only or value-of-grid-cell, and *fails* to learn the value-computing primitives (fill, paint, replace) because the trace dataset's arg_kinds field doesn't store the specific constants.

The takeaway for the world-modeling direction. A learned L1 has to *generalise* across held-out tasks. The lookup table fails to generalise (0% OOD hit rate). The k-NN fails to generalise and barely memorises. The clebran GP **partially generalises**: it can predict the *shape* of every grid→grid primitive's output on a held-out task, and the *value* of pure-geometric primitives, but it cannot predict the *value* of value-computing primitives without seeing the specific constants in the action. **The next step is to extend the trace dataset to record arg values** (not just arg kinds), so the GP can evolve programs that compute the right value; and to expand the program length budget from 3 to 5-7 so the GP can express compositional primitives like `crop` and `hconcat`. The 9.4s evolution time is already within the L2 planner's plausible rollout budget for a one-step planning call (2,500 candidates × 9.4s = 23,500s of precomputation, but the evolved programs are *cached* per primitive so the cost is amortised across the planner's lifetime).

3.3 B5+planner: budget-conditional integration

3 seeds at 500K-1M (the starved-GP regime):

arm	solved	evo-solved	test-exact	mean length
B5 (control, 500K)	9.67	2.67	9.67	1.61
B5+planner (L=1, 500K)	**10.00**	**3.00**	9.67	2.03
B5+planner (L=2, 500K)	**10.00**	**3.00**	9.67	1.70
B5+planner (L=1, 1M)	**10.00**	**3.00**	9.67	2.03

The B5+planner arm is **at parity with B5** at this budget ceiling (per-seed 1M is already at 10/30 in all 3 seeds), with **+0.33 evolution-phase solves** (the planner seed is genuinely contributing to discovery, not just sitting in the population as a no-op) and **1 unique task** (f76d97a5) that B5 misses in any of the 3 seeds. The planner-seeded programs are slightly longer (2.0 vs 1.6 statements), which is the cost of carrying a non-trivial seeded individual.

5 seeds at 4M (the headline configuration), per-seed breakdown:

seed	B5 solved (evo)	B5+P solved (evo)	delta
1	14 (7)	13 (6)	-1
2	12 (5)	13 (6)	+1
3	14 (7)	12 (5)	-2
4	14 (7)	12 (5)	-2
5	13 (5)	12 (4)	-1
mean	**13.40 ± 0.89**	**12.40 ± 0.55**	**-1.00**

5 seeds at 4M (the headline configuration), summary:

arm	solved	evo-solved	test-exact	mean length	unique tasks
B5 (control)	**13.40 ± 0.89**	**6.20**	**12.40**	2.46	**16**
B5+planner (K=1)	12.40 ± 0.55	5.20	11.80	2.37	14

At the headline 4M budget, B5 alone beats B5+planner by 1.0 solve on average (13.4 vs 12.4), with +1.0 evolution-phase solves. The 5 per-seed deltas (-1, +1, -2, -2, -1) span the range from -2 to +1; the worst-case outcome for B5+planner is -2 solves, and the best case is +1 — i.e. B5+planner only "wins" in 1 of 5 seeds, and even then by a single solve. The mean difference (-1.0) is the same magnitude as one standard deviation of either arm (B5 sd=0.89, B5+planner sd=0.55), so the 4M headline is **directionally** (B5 > B5+planner) but not formally tested with a paired-test statistic.

The 2 tasks B5+planner never solves. Across 5 seeds, B5's union of solved tasks is **16 unique tasks**; B5+planner's union is **14 unique tasks**. The intersection is 14 tasks (solved by both arms in at least one seed). The **2 unique-to-B5 tasks** are:

- `0b148d64` — solved by B5 in 1/5 seeds (seed 1)
- `f25fbde4` — solved by B5 in 1/5 seeds (seed 3)

Both are rare-event solves (each requires a specific mutation that the B5+planner's planner-seeded initial individual may be *blocking* — see §4). B5+planner has **zero unique-to-it tasks**; its solved set is a **strict subset** of B5's. The 30% gain from 8x compute (500K → 4M) holds for both arms, but B5 alone scales *slightly better*: at 500K they're within 0.33 of each other; at 4M the gap widens to 1.0.

Budget-conditional reading. The planner seed is useful when the rest of the GP is starved (500K-1M budget, where B5's phase-1 closure saturates at ~7 shallow tasks) and a slight regression at the headline 4M budget (where the phase-1 closure already finds 7-9 tasks and the evolutionary loop finds the rest). The K=3 and K=5 sweeps were not run at 4M (each would add ~80 min); at 500K they were tied with K=1, so the headline at 4M is unlikely to differ.

This is the **first integration result on the clebran rig that is budget-conditional** — paper 1's H1/H2/H3 are uniformly falsified; paper 2's content effects are uniformly null with process effects (order) being the only positive signal; the B5+planner integration is positive at starved budgets and slightly negative at the headline budget. The starved-budget positive result is *useful but not durable* — it tells us the planner seed's value is conditional on the GP's budget, not that the planner seed is a generally-useful addition.

Task-level evidence for the budget-conditional reading. The 30-task frontier subset has a small, identifiable set of tasks where B5 and B5+planner differ:

- `f76d97a5` — the 1 unique task solved by B5+planner but missed by B5 at 500K-1M. A 3x3 / 5x5 / 9x9 demo family where the output is a *mask* of the input: every cell that matches the *center* color becomes 0, every other cell stays the same. The B5 baseline's phase-1 closure enumerates single primitives and short compositions, but the actual solution (`palette` → identify center color → `fill` with the complement) needs a 2-3 statement program whose first statement is non-trivial. The planner's multi-pair, look-ahead search finds this first statement directly. At 4M, B5's evolutionary loop has enough budget to discover the same program, and the unique-task advantage disappears.
- `0b148d64` , `f25fbde4` — the 2 unique tasks solved by B5 alone but missed by B5+planner in any of the 5 4M-eval seeds. Both are large-grid (19x21) tasks where the solution is a *crop* of a specific region. B5's phase-1 closure enumerates the crop primitive and occasionally finds the right `(start, dims)` combination through a single lucky mutation. B5+planner's planner seed occupies a population slot that the GP would otherwise have used for a different initial program, and the planner's grid→grid candidate set doesn't include `crop` with the right parameters for these specific tasks. **The planner seed is *blocking* the rare events B5 would otherwise find** — not by being actively wrong, but by *taking up a slot in the initial population*.

The actual evolved programs. `scripts/inspect_task_programs.py` extracts the evolved programs for the 3 specific tasks from the b5 / b5+planner logs. Side-by-side:

`f76d97a5` (B5+planner unique at 500K-1M, ties at 4M):

```
B5 seed=1   (evolved, len=6, evals=3.13M):
  replace(I, 4, 0)
  mostcolor(x0)
  colorcount(x0, ZERO)
  switch(x0, x2, FIVE)
  mostcolor(x3)
  switch(x3, x4, x2)

B5 seed=2   (evolved, len=5, evals=1.04M):
  mostcolor(I)
  switch(I, FIVE, x0)
  replace(x1, FIVE, ZERO)
  colorcount(x2, x0)
  switch(x2, FOUR, x3)

B5P seed=2  (evolved, len=6, evals=1.50M):
  replace(I, FIVE, FOUR)
  mostcolor(I)
  switch(I, FIVE, ZERO)
  hmirror(ZERO_BY_TWO)             ← library constant
  mostcolor(x0)
  switch(x2, x4, ZERO)

B5P seed=3  (evolved, len=5, evals=0.46M):
  ['abstr', 'b5p-s3_a0'](I)        ← library abstraction
  switch(I, 5, ZERO)
  bottomhalf(x0)
  mostcolor(x2)
  switch(x1, F, x3)
```

B5 and B5+planner converge to *similar* programs on f76d97a5 — they all use `mostcolor` + `switch` + `replace` to identify the centre colour and mask the non-centre cells. The differences (B5P's use of library abstractions and constants, B5's leaner 5-6 statement choreography) are stylistic, not structural. **At 4M, the GP finds the same solution regardless of the planner seed** — the 500K-1M unique advantage of B5+planner comes from finding a *partial* program first (the planner's seed), which the GP then completes via mutation.

`0b148d64` (B5 alone unique at 4M):

```
B5 seed=1   (evolved, len=6, evals=2.91M):
  palette(I)                     ← list of input colors
  last(x0)
  ofcolor(I, 1)                  ← cells with color 1
  ofcolor(I, x1)
  bordering(x3, I)               ← cells of x3 on the border
  subgrid(x3, I)                 ← crop to x3
```

This is a *6-statement* program that finds the colour that forms a border (colour 1), then crops to that region. The planner-seeded individual didn't help here — B5P never solved 0b148d64 in any of 3 seeds. **The crop primitive is the bottleneck**: the planner's grid→grid candidate set doesn't include `subgrid` with the right parameters, so the planner seed never has the right structure to bootstrap a subgrid solution.

`f25fbde4` (B5 alone unique at 4M):

```
B5 seed=3   (evolved, len=4, evals=2.64M):
  ['abstr', 'b5-s3_a3'](I)        ← library abstraction
  hconcat(I, I)                   ← stack I vertically
  mapply(prim:initset, x1)
  upscale(x0, TWO)                ← tile
```

This is a 4-statement *tile* program: the abstraction (presumably a small extracted fragment) produces a unit pattern, hconcat stacks it, mapply fans it out, and upscale tiles the result. Again, B5P never solved this task in any seed. **The planner's candidate set is missing the upscale + hconcat + mapply composition** that B5 found through a lucky library-mining event.

The pattern. The 3 tasks decompose cleanly: f76d97a5 is a *planner-friendly* task (the planner's Hamming-driven multi-pair rollout finds the right first step in a 1-2 statement prefix); 0b148d64 and f25fbde4 are *planner-unfriendly* tasks (they need compositional primitives — `subgrid`, `hconcat`, `mapply`, `upscale` — that the planner's candidate set doesn't include). The planner seed helps in the *first* case (the planner's prefix gives the GP a head start at starved budgets) and *blocks* in the *second* case (the planner's 20-step prefix occupies a population slot the GP could have used for a compositional exploration). At 4M, B5's GP discovers the f76d97a5 solution on its own, but B5+planner still misses the two compositional tasks because the planner's seed is not the right starting point for those searches.

3.4 What flipped the result from "slight null" to "at parity"

The first implementation of the B5+planner arm (pre-§9f) was a *slight null* at 500K: B5+planner 9.33 solves vs B5 9.67, with 0.33 fewer evolution-solves and slightly longer mean programs. The planner-seeded individual was effectively a no-op because **two bugs** in the seed path made the planner plan toward the input grid instead of the target:

1. **`target = input_grid` in `Lineage.seed_with_planner`.** The first version of the planner seed discarded the demo's target (`input_grid, _ = task.train[0]`) and passed `input_grid` as both the state and the target to the planner. The planner found a "no-action" minimum-distance primitive (often `compress` of the input or `vmirror` to itself) and seeded a useless individual. Fixing this required switching to `demos = [(in, target) for in, target in task.train]` — the multi-pair API.

2. **Narrow candidate set.** The first candidate set covered the top *10* grid→grid Hodel primitives. Expanding to the top *20* (adding `dmirror`, `compress`, `normalize`, `trim`, `downscale`, `hupscale`, `upscale`) raised Hodel-first- primitive coverage in the candidate set from 14.4% to 18.5% and iterative solve rate from 2.8% to 3.9% on the trace dataset.

Together: the planner-seeded arm goes from 9.33 (slight null) to 10.00 (parity) at 500K, 3 seeds, with +0.33 evolution-phase solves and 1 unique task B5 misses.

4. Discussion

Why the planner helps at starved budgets (500K-1M). B5's phase-1 seeding is a *systematic closure* over short programs: it enumerates single statements and short compositions, evaluates them, and keeps the best. At a budget where the frontier subset ceiling is ~10/30, the phase-1 closure already finds the 7 *shallow* tasks (the depth-1 / depth-2 solvable subset), and the remaining 3 solves come from the evolutionary loop's depth-3 exploration. The planner seed contributes a *non-trivial program that B5's depth-1 closure did not generate* — typically a length-2 or length-3 sequence like `vmirror` + `hconcat` or `fill` + `paint`. This pre-computed, non-shallow individual gives the evolutionary loop a *better starting point* for the depth-3 search, and the +0.33 evolution-phase solves is the visible consequence. The unique task f76d97a5 is the case where the planner's length-2 program happened to be the solution B5's depth-1 closure missed.

Why the planner regresses at the headline budget (4M). At 4M, the phase-1 closure already finds 7-9 tasks and the evolutionary loop finds the rest. The planner seed is now *redundant* — the GP would have found the planner's program itself (or a better one) through its own search. Worse, the planner seed is *non-trivial* (length 2.0-2.5 vs B5's 1.6-2.5), which means it occupies a population slot that the GP could have used for a different initial program. The result: B5+planner solves 1.0 fewer task on average, with 1.0 fewer evolution-phase solves. **The B5+planner's solved set is a strict subset of B5's solved set** — B5 solves 2 tasks that B5+planner never solves in any of the 5 seeds (0b148d64 and f25fbde4, each a single-seed rare-event solve), while B5+planner solves *zero* unique tasks that B5 doesn't also solve. The worst-case delta is -2 solves (seeds 3 and 4); the best-case delta is +1 solve (seed 2). The planner seed isn't just neutral at 4M — it actively *blocks* the rare events B5 would otherwise find.

Where the planner can't help (yet). The B5+planner *ceiling at 4M is 13/30* vs B5's 14/30 — the planner seed is not the ceiling-breaker. The planner is *one* individual in a 100-individual population; scaling to top-K planner seeds (e.g. the top-5 candidates from the multi-pair evaluation) was tested at 500K (top-K=3,5 tied with K=1) but not at 4M. The planner's 3.4% multi-pair solve rate (constrained by the candidate set and the Hamming metric) caps how often the planner can directly produce a solution; the rest of the time, the planner's contribution is via *seeding the search* rather than solving outright. The two levers that could break the 13/30

ceiling are: (i) **object- aware distance** (added as a configuration option, but null result at 4M seed=1: structural is 9.33 vs hamming's 10.0), and (ii) **re-arc-generated trace candidates**, which would expand the candidate set with primitives inferred from re-arc's per-task generators, not just Hodel's reference solvers.

The 4M result is a *negative* result for the planner seed as a general-purpose integration. It is, however, a *useful* result in a different sense: it tells us precisely the *condition* under which the planner seed helps. The planner seed is a *budget-conditional intervention* — useful when the GP is starved, neutral-or-negative when the GP has enough budget. This is the kind of result the clebran rig is designed to make believable: a falsified integration at the headline configuration, with a precise budget threshold for when the integration flips from positive to neutral-or-negative. The threshold appears to be the point at which B5's phase-1 closure saturates the frontier subset (around 1M-2M evals).

The perfect L1 is the asset. The result is a clean demonstration that the clebran interpreter's perfect L1 is a useful planning substrate. The k-NN baseline (§2.2) shows the *learned* L1 is not yet a productive replacement, but the *perfect* L1 is. The learned L1's job is to be a fast approximation of the perfect L1, with a tolerable accuracy loss in exchange for rollout speed — and the clebran rig is the right test bed for measuring that accuracy / speed trade-off against an L2 planner using the perfect L1 as the ground truth.

The world-modeling direction is exactly what ARC-AGI-3 was designed to test. ARC-AGI-3 (arXiv:2603.24621) lists four required capabilities: *Exploration, Modeling, Goal-Setting, Planning and Execution*. The *Modeling* capability is "the ability to build a world model that represents how the environment works and can predict the results of unseen actions." The clebran L2 planner is exactly that: a model (the clebran interpreter) that predicts the results of unseen actions, used to plan. The substrate is ARC-AGI-1; the direction transfers to ARC-AGI-3 with no substrate change.

5. Limitations

Sample size. The 4M-eval sweep is 5 seeds (vs paper 1's 20-seed headline). The 5 per-seed deltas (-1, +1, -2, -2, -1) have mean **-1.0** and standard deviation **1.22**. The SE on the mean delta is **$1.22/\sqrt{5} = 0.55$**, so the per-seed mean delta is at **~1.8 SE** below zero — a directional result *just below* the 2-SE descriptive bar the rig uses (paper 1, §4.5). 4/5 deltas are negative, 1/5 is positive (+1, seed 2), and no seed shows a tie or a +2. The B5 win is not yet a rigorously-rejected null at this seed count, but the per-seed direction is consistent. The 20-seed sweep is the natural extension if a formal 2-SE-bar reading is wanted; given the per-seed direction, the headline pattern (B5 alone beats B5+planner at 4M) is unlikely to reverse at 4x more seeds.

K=3 and K=5 not run at 4M. Each would add ~80 min; the 500K pilot (K=1, 3, 5) was tied at 10.00, so the 4M result for K=3 and K=5 is unlikely to differ from K=1 in a meaningful way (the

mean is already 12.4 for K=1, and the per-seed variance is 0.55). The K=1 headline is the binding one.

Distance metric. The planner uses Hamming distance by default. Object-aware distances (component, color_set, structural) are implemented as configuration options but tested only at 4M seed=1: the structural distance gave 9.33 vs the default hamming's 10.0. The structural distance is useful for diagnostic experiments but not the right default for the integration. See §4.

Candidate set coverage. The planner's candidate set covers the top 20 grid→grid Hodel primitives, missing 2-arg / 3-arg compositions with smart defaults (e.g. `ofcolor` + `recolor`, `cellwise(state, vm, 0)`). Re-arc-generated trace candidates would expand this set dramatically. The 3.4% multi-pair solve rate is the planning ceiling with the current candidate set.

Substrate scope. The result is on the clebran substrate (typed evolutionary program synthesis on ARC-AGI-1, 30-task frontier subset). Transfer to ARC-AGI-3, to neural program synthesis, or to other DSL settings is an empirical question for each.

The L1 baseline is the wrong model class. The 0% OOD exact-match / 5% in-distribution exact-match k-NN result is a *diagnostic*, not a ceiling. Two programmatic learned L1s (§3.2.5) show the ceiling is reachable but with split specialisation: the **lookup-table L1** is the value-accurate L1 (25.5% OOD exact-match, 4.20 mean Hamming on shape-matching predictions) but cannot generalise (0% OOD hit rate); the **GP-evolved L1** is the shape-accurate L1 (52.7% OOD shape-match, 18.7% OOD exact-match) and can learn pure geometric primitives perfectly (rot90, hmirror, vmirror, dmirror, cmirror) but fails to learn value-computing primitives (fill, paint, replace) without the specific color constants in the action. The learned L1 is partially built; the next push is to extend the trace dataset to record arg *values* (not just arg kinds) and to expand the GP's program-length budget.

The B5 baseline is the relevant control, not a no-planner arm. The B5+planner arm's purpose is to test whether a planner-seeded individual helps B5's central-library search. A no-planner arm (i.e. B5 alone) is the B5 control, not a separate arm. The 4M null result is specifically a B5 vs B5+planner comparison, not a question about whether the planner seed helps *any* arm.

6. Conclusion

The world-modeling direction for clebran is **alive but budget-conditional**. The trace dataset is built (16,149 rows, 1,302 demo pairs, 6.3 MB); the L1 baseline is characterised (k-NN is the wrong model class, 0% OOD / 5% in-dist exact-match); two programmatic learned L1s are built and compared head-to-head (lookup-table: 25.5% OOD exact-match, 0% OOD hit rate, value-accurate but not generalising; GP-evolved: 18.7% OOD exact-match, 52.7% OOD shape-match, perfect on pure-geometric primitives, shape-only on value-computing primitives, 9.4s total

evolution); the L2 planner is built (multi-pair, lookahead, loop-avoiding, 35.9% one-step improve, 3.9% iterative solve); the top-K planner seed is built (`--planner-top-k K`, K=3 and K=5 tied with K=1 at 500K); the object-aware distance metrics are built (component, color_set, structural — but null at the integration level); and the **B5+planner arm produces a budget-conditional result**: parity at 500K-1M (10.0 vs 9.67, +0.33 evo, 1 unique task f76d97a5) and a slight regression at the headline 4M budget (12.4 vs 13.4, -1.0 evo, -2 unique tasks).

The headline finding is budget-conditionality. The planner seed is useful when the GP is starved (500K-1M) and a slight regression when the GP has enough budget to do the work itself (4M). This is the *first* result on the clebran rig that is *conditional* on the GP's budget — paper 1's H1/H2/H3 are uniformly falsified across budgets; paper 2's content effects are uniformly null with process effects (order) being the only positive signal; the B5+planner integration is positive at starved budgets and slightly negative at the headline budget.

The world-modeling direction's value is in the **substrate it provides** for the next push: the trace dataset (16K rows) is the labelled transition corpus for a learned L1; the L2 planner is a reusable multi-pair planning module; and the clebran interpreter is the perfect L1 that any learned L1 has to approximate. The B5+planner integration specifically is a *null result at scale*, and the next push is the learned L1 itself — a small neural or programmatic approximation of the interpreter that, once it generalises, replaces the perfect- but-slow interpreter as the planner's rollout engine. Re-arc- generated trace candidates and object-aware distance are the natural supporting improvements; the direction is exactly what ARC-AGI-3 (arXiv:2603.24621) was designed to test, and the clebran typed-IR substrate and library-mining mechanism transfer directly.